Ders 22 Çalışma Özeti

Ders 22 Çalışma Özeti

Genel Konular

- Vize sorularının çözümü: yığın, kuyruk, bağlı liste, heap, özyineleme, sıralama, ikili arama ağacı soruları ve kod sorusu.
- En küçük kapsayan ağaç için Prim algoritması: iki örnek.
- Prim ve Kruskal karşılaştırması.
- En kısa yol problemi için Dijkstra algoritması: elle çözüm ve matrisle çözüm.

Hocanın Özellikle Vurguladığı Kısımlar

- Kod sorusunda en sık hata algoritmanın akışındaydı. Önce sayfanın listede olup olmadığına bakılmalıydı.
- Bağlı listede bir düğümü bulduktan sonra onu tekrar aramak için listeyi baştan gezmeyin. Gezerken `prev` ve `current` adreslerini tutun.
- Hoca `while` içinde `break` kullanılmasını istemiyor. `while` zaten adım sayısı belli olmayan döngüler içindir. Fonksiyondan `return` ile çıkmak sorun değil.
- Bağlı liste istenen soruda dizi kullanılmamalı.
- Prim'de bir sonraki düğüm, elde olan düğümlerden ulaşılabilen en küçük kenarla seçilir. Ziyaret edilmiş düğüme giden kenar alınmaz.
- Dijkstra'da bir sonraki adımda her zaman en küçük uzaklığa sahip, henüz işlenmemiş düğümle devam edilir. Alfabetik sırayla devam etmek yanlış sonuç verebilir.

Kısa Tekrar Notları

- Prim: herhangi bir düğümden başlar. Ağaçtan dışarı çıkan en küçük kenarı ekler.
- Prim karmaşıklığı: komşuluk matrisiyle $O(V^2)$, komşuluk listesi ve ikili heap (min heap) ile $O(E \log V)$.
- Kruskal karmaşıklığı: $O(E \log V)$.
- Prim yoğun graflarda, Kruskal seyrek graflarda daha hızlıdır.
- Prim öncelikli kuyruk (priority queue), Kruskal ayrık küme (disjoint set) kullanır.
- Eşit ağırlıklı kenarlar varsa birden fazla MST olabilir; toplam ağırlık aynıdır.
- Dijkstra: başlangıçta kaynak dışındaki tüm uzaklıklar ∞. Her adımda en küçük uzaklıklı düğüm seçilir ve komşularının uzaklığı güncellenir (relaxation).
- Güncelleme kuralı: $d[v] > d[u] + w(u, v)$ ise $d[v] = d[u] + w(u, v)$.

Detaylı Açıklamalar

Vize Sorularının Çözümü

Hoca soruları Word belgesi üzerinden tek tek çözdü.

Soru	Konu	Cevap
1	$n = 20$ için yığın kullanan kod	Sayının ikilik karşılığını yazar: `10100`
2	Kuyruktan yığına, yığından kuyruğa aktarım (5, 2, 1, 4)	Çıktı: 4, 1, 2, 5

Soru	Konu	Cevap
3	`x.next = x.next.next;`	x'ten sonraki düğüm listeden çıkarılır (silinir). `free` yapılmadığı için bellek serbest kalmaz.
4	Max heap'te en büyük elemanı silip heapify	Karşılaştırmaya giren sayılar: 19, 37, 35, 34, 36, 14, 17
5	Base condition olmayan özyinelemeli fonksiyon	Sonsuz çalışma durumu oluşur (D şıkkı)
6	Yalnız ilk elemanın adresi tutulan, tek yönlü bağlı listeyle kuyruk	`enqueue` $O(n)$, `dequeue` $O(1)$
7	Tüm elemanları aynı olan dizi	Selection sort $O(n^2)$, insertion sort $O(n)$
8	Dairesel bağlı listenin başına ekleme (en kötü durum)	Son düğümün adresi tutulursa $O(1)$
9	İkili arama ağacına ekleme (en kötü durum)	$O(n)$
10–11	`mystery` fonksiyonu, 1 3 5 2 4 ve 1 3 5 4 2 çıktıları	1 3 5 2 4 oluşamaz; 1 3 5 4 2 oluşabilir
12	Sırasız A_n ve sıralı B_m dizilerinin ortak elemanları	A'nın her elemanını B'de ikili arama ile ara: $O(n \log m)$
13–15	Harflerle maskelenmiş BST'de "X'ten büyük ilk değer"	A için J, B için E, C için K
16	Ağacın yüksekliği	4
17	Yüksekliği bozmadan eklenebilecek eleman	18

Önemli açıklamalar: - **Soru 1:** 20'nin 2'ye göre kalanları sırayla yığına atılır: 0, 0, 1, 0, 1. Yığından çıkış ters sırada olur: `10100`. - **Soru 4:** Kökteki 38, sondaki 19 ile yer değiştirir. 19 aşağı iner: - 37 ile 35 karşılaştırılır, 37 yukarı çıkar. - 34 ile 36 karşılaştırılır, 36 yukarı çıkar. - 14 ve 17, 19'dan küçük. İşlem biter. - En sık yapılan hata: 16 ve 15'i de karşılaştırmaya katmak. - **Soru 6:** Yalnız baş adresi tutuluyorsa sona eklemek için tüm liste gezilir. - **Soru 7:** Selection sort'un iç döngüsünde erken çıkış yok. Insertion sort'ta kaydırma olmadığı için her eleman tek karşılaştırmayla yerinde kalır. - **Soru 8:** Baş yerine kuyruk (son düğüm) adresini tutun. Son düğümün `next`'i zaten başı gösterir. - **Soru 9:** Sıralı gelen elemanlar ağacı tek tarafa yaslar (degenerate tree). - **Soru 10–11:** Sayılar ya hemen yazılıyor ya da yığına atılıyor. Yığındaki 2 ve 4 ters sırada çıkar. Bu yüzden 2'den önce 4 yazılır. - **Soru 12:** - $n \gg m$ ifadesi çeldiricidir. - Belirleyici olan dizilerin sıralı olup olmamasıdır. - İkili arama yalnız sıralı dizide yapılabilir. - **Soru 13–15:** Bir düğümden büyük ilk değeri (inorder ardılını) bulmak için sağ çocuğa geçin. Sonra sola gidilebildiği kadar gidin. - **Soru 17:** - Yüksekliği 4 olan dolu bir ağaçta 0–4. seviyelerde $1 + 2 + 4 + 8 + 16 = 31$ düğüm olur. - Ağaçta 13 düğüm var. $31 - 13 = 18$ (bu ara adım derste söylenmedi; ilk üç seviyenin dolu olduğu varsayımıyla hocanın sayımından çıkarıldı). - Hocanın sayımı: son seviyeye 15, önceki seviyedeki boş yerlere 3 eleman.

Kod Sorusu: Tarayıcı Sayfa Listesi

Kurallar: - İlk kez bakılan sayfanın kod numarası listenin başına eklenir. - Eleman sayısı N'yi geçerse en sondaki sayfa silinir. - Daha önce bakılmış sayfanın sayacı artırılır. - Sayaç K'ye ulaşırsa sayfa listenin başına taşınır ve sayacı sıfırlanır.

Doğru akış: 1. Listeyi `current != NULL` ve `current->kod != aranan` olduğu sürece gez. `prev` adresini de tut. 2. Sayfa bulunduysa: - Sayaç K ise `prev->next = current->next` ile düğümü çıkar. Düğümü başa ekle, sayacı sıfırla. - Değilse yalnız sayacı artır. 3. Sayfa bulunamadıysa: - Yeni düğümü başa ekle, eleman sayısını artır. - Eleman sayısı N'yi geçtiyse sondan bir önceki düğümün `next` alanını `NULL` yap. Kuyruk (tail) adresi de tutulabilir.

Sık yapılan hatalar: - Bulunan düğümü taşımak için listeyi bir kez daha gezmek. - N ve K değerlerini okuyup hiç kullanmamak. - Bağlı liste yerine dizi kullanmak. - Adımların sırasını yanlış kurmak.

Prim Algoritması: Birinci Örnek

Yönsüz, ağırlıklı graf: A-B 3, A-C 6, B-C 2, B-D 1, B-E 5, C-D 4, D-E 2.

Adım	Ağaçtaki düğümler	Seçilen kenar	Ağırlık
1	A	A-B	3
2	A, B	B-D	1
3	A, B, D	B-C (D-E ile eşit; alfabetik sıra)	2
4	A, B, C, D	D-E	2

- MST: A-B, B-D, B-C, D-E. Toplam: $3 + 1 + 2 + 2 = 8$.
- Uçları ağaçta olan kenarlar (A-C 6, C-D 4) artık aday değildir.

Prim'in Karmaşıklığı

- Komşuluk matrisiyle: $O(V^2)$.
- Komşuluk listesi ve min heap ile: $O(E \log V)$.
- Her adımda en küçük kenar seçildiği için en küçük elemana hızlı erişen yapı gerekir. Bu yapı min heap'tir.
- Heap'ten eleman alındıktan sonra yapılan heapify işlemi log çarpanını getirir.

Prim Algoritması: İkinci Örnek

Geçen haftaki Kruskal örneğindeki 9 düğümlü graf kullanıldı.

Kenarlar: 0-1 4, 0-7 8, 1-2 8, 1-7 11, 2-3 7, 2-5 4, 2-8 2, 3-4 9, 3-5 14, 4-5 10, 5-6 2, 6-7 1, 6-8 6, 7-8 7.

Ziyaret listesi tutulur. Başlangıçta hiçbir düğüm ziyaret edilmemiştir.

0'dan başlayınca seçim sırası:

Adım	Seçilen kenar	Ağırlık
1	0-1	4
2	0-7 (1-2 ile eşit)	8
3	7-6	1
4	6-5	2
5	5-2	4
6	2-8	2
7	2-3	7
8	3-4	9

- Toplam: $4 + 8 + 1 + 2 + 4 + 2 + 7 + 9 = 37$. Kruskal ile aynı sonuç.
- 8'den başlayınca 0-7 yerine 1-2 (8) kenarı seçildi. Toplam yine 37.
- Sebep: 0-7 ve 1-2 kenarlarının ağırlığı eşit (8). Hangisi seçilirse seçilsin toplam değişmez.
- Bir öğrencinin özeti: "MST değişebilir ama maliyet sabit kalır." Hoca doğruladı.
- Hocanın sorusu: Tüm kenar ağırlıkları farklıysa birden fazla MST olabilir mi? Cevap ders sonrası notlarda.

Prim ve Kruskal Karşılaştırması

Prim	Kruskal
Herhangi bir düğümden başlar.	En küçük ağırlıklı kenardan başlar.

Prim	Kruskal
Bir düğümü birden fazla kez kontrol eder.	Bir düğümü bir kez işler.
$O(V^2)$ ya da $O(E \log V)$	$O(E \log V)$
Bağlı bir bileşen üretir; yalnız bağlı graflarda çalışır.	Ara adımlarda orman üretebilir; bağlı olmayan graflarda da çalışır.
Yoğun graflarda daha hızlı.	Seyrek graflarda daha hızlı.
Gerçeklemesi daha zor.	Gerçeklemesi daha kolay.
Öncelikli kuyruk kullanır.	Ayrık küme (disjoint set) kullanır.

- "Bir düğümü birden fazla kez gezer" ifadesinin anlamı:
  - Prim her adımda ağaçtan çıkan tüm aday kenarlara bakar.
  - Bir kenar, ziyaret edilmiş bir düğüme gidiyor olabilir.
  - Bu yüzden aynı düğüm için tekrar tekrar "ziyaret edildi mi" kontrolü yapılır.
- Kruskal kenarları sıraladığı için kenar sayısı az olan seyrek graflarda daha avantajlıdır.

Dijkstra Algoritması

- Amaç: bir kaynak düğümden diğer tüm düğümlere en kısa yolu bulmak.
- Prim'den farkı:
  - Prim tüm grafı en az toplam maliyetle bağlar.
  - Dijkstra kaynaktan her düğüme ayrı ayrı en kısa yolu bulur.
- Prim'deki bakış açısı kullanılır: her adımda en küçük değerli düğümle devam edilir. Bu da açgözlü bir yaklaşımdır.

Örnek graf (yönlü):

Kenar	Ağırlık
A→B	3
A→C	6
A→D	4
B→C	2
B→E	3
C→E	2
C→F	3
D→C	2
D→F	5
D→G	7
E→F	1
F→G	1

Matrisle çözüm. Her satır o adımda işlenen düğümü, sütunlar hedef düğümleri gösterir.

İşlenen	B	C	D	E	F	G
A	**3**	6	4	∞	∞	∞
B		5	**4**	6	∞	∞
D		**5**		6	9	11
C				**6**	8	11
E					**7**	11
F						**8**

- B işlenince C: $3 + 2 = 5 < 6$, güncellenir. E: $3 + 3 = 6$.
- Sıradaki düğüm C değil D'dir, çünkü D'nin değeri (4) en küçüktür.
- D işlenince C: $4 + 2 = 6$, 5'ten küçük değil. F: $4 + 5 = 9$. G: $4 + 7 = 11$.
- C işlenince E: $5 + 2 = 7$, güncellenmez. F: $5 + 3 = 8 < 9$, güncellenir.
- E işlenince F: $6 + 1 = 7 < 8$, güncellenir.
- F işlenince G: $7 + 1 = 8 < 11$, güncellenir.

Sonuç (A'dan en kısa uzaklıklar):

Düğüm	Uzaklık	Yol
B	3	A→B
C	5	A→B→C
D	4	A→D
E	6	A→B→E
F	7	A→B→E→F
G	8	A→B→E→F→G

- Sonuçlar matrisin köşegeninden okunmaz. Her sütunda ∞'dan önce görülen son değer okunur. Matris bu yüzden aşağıya doğru taşınır.
- Başka bir kaynaktan başlanırsa bazı düğümlere hiç ulaşılamayabilir. Örneğin E'den yalnız F'ye ve G'ye ulaşılır; A, B, C ve D'ye yol yoktur.

Hocanın araştırma soruları: - En kısa yolun hangi düğümlerden geçtiği nasıl bulunur? - Negatif ağırlıklı kenarlarda Dijkstra çalışır mı? Çalışmazsa hangi algoritma kullanılır? - Başka en kısa yol algoritmaları nelerdir? - En büyük ağırlıklı yolu gerektiren problemler var mıdır?

Ders sonrası notlar

- **Soru 12:** Altyazıda bir yerde "$M \log M$" geçiyor. Doğru karmaşıklık $O(n \log m)$'dir: n eleman, her biri için m elemanlı dizide ikili arama.
- **Tek MST:** Tüm kenar ağırlıkları birbirinden farklıysa MST tektir. Bu yüzden "8 + 3 = 7 + 4" gibi toplam eşitlikleri ikinci bir MST oluşturamaz. Birden fazla MST ancak eşit ağırlıklı kenarlar varken mümkündür.
- **Dijkstra ve yön:** Hoca yolların yönlü kabul edildiğini söyledi. Dijkstra yönsüz graflarda da çalışır; yönsüz kenar iki yönlü kenar gibi düşünülür.
- **Dijkstra ve negatif ağırlık:** Negatif ağırlıklı kenarlarda Dijkstra doğru sonuç vermeyebilir. Bu durumda Bellman-Ford kullanılır. Tüm düğüm çiftleri için Floyd-Warshall kullanılabilir.
- **Yolun kendisi:** Her güncellemede önceki düğüm (`parent`/`prev`) bir dizide saklanır. Yol, hedef düğümden kaynağa doğru bu dizi izlenerek bulunur.
- **Maksimum akış:** Maksimum akış (maximum flow) problemi "en uzun yol" problemi değildir. Bir ağdan kaynaktan hedefe geçebilecek en fazla akışı bulur (örneğin Ford-Fulkerson). En uzun yol problemi ayrı bir problemdir.
- **Dijkstra'nın karmaşıklığı:** Matrisle $O(V^2)$, ikili heap ve komşuluk listesiyle $O((V + E) \log V)$.
- **Prim birinci örnek:** Hoca kenarları okurken bir yerde "D'den E'ye 5", bir yerde "B'den E'ye 2" dedi. Doğrusu B-E 5, D-E 2'dir (tablodaki gibi). MST toplamı 8'dir.
- **Dijkstra elle çözüm:** Hoca elle çözümde düğümleri alfabetik sırayla işledi (C'yi D'den önce). Dijkstra'da her adımda henüz işlenmemiş en küçük uzaklıklı düğüm seçilir: A, B, D, C, E, F, G. Bu örnekte sonuç değişmedi (python ile doğrulandı), ama alfabetik sıra genel olarak yanlış sonuç verebilir. Hoca dersin sonunda doğru sırayı kendisi de vurguladı.
- **E'den ulaşılabilen düğümler:** Hoca "E'den B'ye gidebilirsiniz" dedi. Yönlü grafta E'den çıkan tek kenar E→F'dir; E'den yalnız F (1) ve G (2) düğümlerine ulaşılır.
- **Prim örneği:** Hoca ikinci örnekte bir adımda yanlış kenarı işaretledi. Bir öğrenci uyardı, hoca düzeltti. Tablo düzeltilmiş hâliyle verildi.
- **`break` kullanımı:** `while` içinde `break` kullanmamak hocanın tercih ettiği bir stil kuralıdır. C dilinde hata değildir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.